Lab & Exam Study Materials

Note: While most of the routing, middleware, and MongoDB queries are drawn directly from your provided sources (like Express request/response methods and MongoDB operators), specific code implementations for things like File Uploads (Q8) and Event Handling (Q8) extend beyond your provided text. Supplemented with standard Node.js practices, which you should verify with your actual lab manual.

QUESTION 1

AIM

To create a simple Node.js web server that reads data from the URL path, and to write a MongoDB command to insert multiple products into the inventory collection.

PROCEDURE

1. Import the `http` module to create a server.
2. Parse the `req.url` to read the path data.
3. Start the server on port 3000.
4. For MongoDB, use the `insertMany()` method passing an array of documents.

NODE.JS PROGRAM

```
const http = require('http');
const server = http.createServer((req, res) => {
    res.statusCode = 200;
    res.setHeader('Content-Type', 'text/plain');
    // Read and display data from the URL path
    res.end(`You accessed the path: ${req.url}`);
});
server.listen(3000, () => {
    console.log('Server is running on port 3000');
});
```

MONGODB QUERY

```
db.inventory.insertMany([
    { name: "Laptop", quantity: 50, price: 800 },
    { name: "Mouse", quantity: 150, price: 20 },
    { name: "Keyboard", quantity: 100, price: 30 }
])
```

OUTPUT

```
Node.js: You accessed the path: /products (when visiting localhost:3000/products).
MongoDB: { acknowledged: true, insertedIds: [ ObjectId("..."), ... ] }.
```

RESULT

The web server successfully reads the URL path, and multiple records were successfully inserted into the MongoDB collection.

QUESTION 2

AIM

To write a Node.js program that logs the request method and URL while handling unsupported methods, and to write MongoDB commands to find specific documents.

PROCEDURE

1. Create an Express.js application.
2. Create a middleware function using `app.use()` to log `req.method` and `req.url`.
3. Add a condition to return a 405 error if the method is unsupported.
4. For MongoDB, use the `findOne()` or `find()` methods with conditional operators like `$lt`.

NODE.JS PROGRAM

```
const express = require('express');
const app = express();

// Middleware to log request method and URL
app.use((req, res, next) => {
    console.log(`Request Method: ${req.method}, Request URL: ${req.url}`);

    // Handle unsupported HTTP methods
    if(req.method !== 'GET' && req.method !== 'POST') {
        return res.status(405).send('Method Not Allowed');
    }
    next();
});

app.get('/', (req, res) => res.send('Welcome!'));
app.listen(3000, () => console.log('Server running on port 3000'));
```

MONGODB QUERIES

```
a) db.locations.findOne({ city: "Delhi" })
b) db.employees.find({ salary: { $lt: 50000 } })
```

OUTPUT

> **Node.js Console:** Request Method: GET, Request URL: /
> **MongoDB:** Displays the specific document for Delhi, and lists all employee documents matching

```
the salary condition.
```

RESULT

The application logs incoming requests and filters unsupported methods, while MongoDB successfully filters documents based on the specified conditions.

QUESTION 3

AIM

To use the Node.js fs module to write and read file contents with error handling, and to update specific documents in MongoDB.

PROCEDURE

1. Import the `fs` (File System) module.
2. Use `fs.writeFile` to write data and `fs.readFile` to read it back, checking for the `err` object in the callback.
3. For MongoDB, use `updateOne()` and `updateMany()` with the `$set` operator to modify documents.

NODE.JS PROGRAM

```
const fs = require('fs');

// Write data to a file
fs.writeFile('data.txt', 'Hello, this is file data!', (err) => {
    if (err) return console.error('Write Error:', err); // Handle write error
    console.log('File written successfully.');

    // Read the contents of the file
    fs.readFile('data.txt', 'utf8', (err, data) => {
        if (err) return console.error('Read Error:', err); // Handle read error
        console.log('File Contents:', data);
    });
});
```

MONGODB QUERIES

```
a) db.employees.updateOne({ name: "Rahul" }, { $set: { department: "Sales" } })
b) db.inventory.updateMany({ status: "out-of-stock" }, { $set: { quantity: 0 } })
```

OUTPUT

```
Node.js: File written successfully. File Contents: Hello, this is file data!
MongoDB: { matchedCount: 1, modifiedCount: 1 }.
```

RESULT

File I/O operations were executed with proper error handling, and target documents in MongoDB were accurately updated.

QUESTION 4

AIM

To write an Express.js program using third-party middleware and a POST route, and to write MongoDB commands to delete specific documents.

PROCEDURE

1. Import Express and a third-party middleware like `body-parser` (or use built-in `express.json()`).
2. Define an `app.post()` route to receive and send back `req.body` data.
3. For MongoDB, use `deleteOne()` and `deleteMany()` methods with proper filtering conditions.

EXPRESS.JS PROGRAM

```
const express = require('express');
const bodyParser = require('body-parser'); // Third-party middleware
const app = express();

// Use third-party middleware to parse JSON
app.use(bodyParser.json());

// POST route to receive user data
app.post('/userdata', (req, res) => {
    const userData = req.body;
    res.send(`Received user data: ${JSON.stringify(userData)}`);
});

app.listen(3000, () => console.log('Server running'));
```

MONGODB QUERIES

```
a) db.tickets.deleteOne({ status: "closed" })
b) db.products.deleteMany({ rating: { $lt: 3 } })
```

OUTPUT

```
Express: Received user data: {"name":"John","age":30}
MongoDB: { deletedCount: 1 }.
```

RESULT

Express successfully parsed POST data using middleware, and MongoDB accurately removed the targeted records.

QUESTION 5A

AIM

To create an Express.js program that redirects requests and sends responses based on query parameters, and to find specific records in MongoDB using array operators.

PROCEDURE

1. Use `res.redirect()` to forward requests to a new URL.
2. Access `req.query` to evaluate URL parameters and send different `res.send()` outputs.
3. For MongoDB, use `find()` to match exact strings, and the `$in` array operator to match tags.

EXPRESS.JS PROGRAM

```
const express = require('express');
const app = express();

// Redirect a request to another route
app.get('/old-route', (req, res) => {
    res.redirect('/new-route');
});

// Send different responses based on query parameters
app.get('/new-route', (req, res) => {
    const type = req.query.type;
    if (type === 'admin') {
        res.send('Welcome, Admin!');
    } else {
        res.send('Welcome, Guest!');
    }
});

app.listen(3000, () => console.log('Server running'));
```

MONGODB QUERIES

```
a) db.students.find({ class: "10A" })
b) db.posts.find({ tags: { $in: ["technology"] } })
```

OUTPUT

```
Express: Requesting /old-route?type=admin redirects and displays Welcome, Admin!.
MongoDB: Returns JSON documents matching the queries.
```

RESULT

The application handles routing redirects and query parameters correctly, while the database fetches specific array-based elements.

QUESTION 5B

AIM

To write an Express.js program that sets a status code and parses data, and to use MongoDB to query age ranges and regular expressions.

PROCEDURE

1. Use built-in middleware `express.json()` to parse request data.
2. Use `res.status(code).send()` to chain a status code with a response.
3. For MongoDB, use `$gte` and `$lte` for ranges, and the `$regex` operator to find sub-strings.

EXPRESS.JS PROGRAM

```
const express = require('express');
const app = express();

// Parsing data from request
app.use(express.json());

app.post('/data', (req, res) => {
    const parsedData = req.body;
    // Setting the status code and responding
    res.status(201).send(`Data created: ${JSON.stringify(parsedData)}`);
});

app.listen(3000, () => console.log('Server running'));
```

MONGODB QUERIES

```
a) db.students.find({ age: { $gte: 18, $lte: 25 } })
b) db.books.find({ title: { $regex: ".*MongoDB.*" } })
```

OUTPUT

```
Express: Responds with HTTP Status 201 and Data created: {...}
MongoDB: Returns documents where age is 18-25 and titles containing "MongoDB".
```

RESULT

Data was successfully parsed and acknowledged with a custom status code. MongoDB fetched ranged and string-matched data.

QUESTION 6

AIM

To set status codes and display messages in an Express browser response, and to execute update commands in MongoDB.

PROCEDURE

1. Define a GET route in Express.
2. Use `res.status(200).send()` to output HTML/text to the browser.
3. For MongoDB, use `updateOne()` for a single record and `updateMany()` to modify all documents in a collection.

EXPRESS.JS PROGRAM

```
const express = require('express');
const app = express();

app.get('/status', (req, res) => {
    // Setting status code and displaying message in browser
    res.status(200).send('<h1>Operation Successful!</h1>');
});

app.listen(3000, () => console.log('Server running'));
```

MONGODB QUERIES

```
a) db.students.updateOne({ name: "Alice" }, { $set: { age: 25 } })
b) db.employees.updateMany({}, { $set: { status: "active" } })
```

OUTPUT

```
Express: Browser displays a large heading "Operation Successful!".
MongoDB: { matchedCount: X, modifiedCount: Y }.
```

RESULT

Status codes and browser messages were rendered effectively, and target collections were bulk-updated.

QUESTION 7

AIM

To implement multiple middlewares and filter paths using URL prefixes in Express, and to update document fields in MongoDB.

PROCEDURE

1. Create two middleware functions (e.g., logger1, logger2).
2. Use `app.use('/api', ...)` to apply these middlewares only to paths with the /api prefix.
3. For MongoDB, use `$set` inside an `updateOne()` and `updateMany()` query.

EXPRESS.JS PROGRAM

```
const express = require('express');
const app = express();

// Implementation of 2 middlewares
const middleware1 = (req, res, next) => { console.log('Mid 1'); next(); };
const middleware2 = (req, res, next) => { console.log('Mid 2'); next(); };

// Filtering paths using URL prefix ('/api')
app.use('/api', middleware1, middleware2);

app.get('/api/users', (req, res) => {
    res.send('Filtered Route Accessed');
});

app.listen(3000, () => console.log('Server running'));
```

MONGODB QUERIES

```
a) db.students.updateOne({ name: "Bala" }, { $set: { phone: "99**" } })
b) db.employees.updateMany({}, { $set: { status: "active" } })
```

OUTPUT

```
Express: Terminal prints Mid 1 and Mid 2 only when visiting /api/users.
MongoDB: { matchedCount: 1, modifiedCount: 1 }.
```

RESULT

Middleware was successfully restricted to specific URL prefixes.

QUESTION 8

AIM

To write a Node.js program handling file uploads and custom events, and to execute greater-than and regex queries in MongoDB.

PROCEDURE

1. Import the `events` module to create and handle a custom event.
2. Simulate a file upload mechanism handling data chunks.
3. For MongoDB, use the `$gt` comparison operator and the `$regex` evaluation operator.

NODE.JS PROGRAM

```
const http = require('http');
const EventEmitter = require('events');

// Event handling
const myEmitter = new EventEmitter();
myEmitter.on('fileUploaded', () => console.log('Custom Event: File upload complete!'));

const server = http.createServer((req, res) => {
    if (req.method === 'POST' && req.url === '/upload') {
        let fileData = '';
        // File upload operation (reading chunks)
        req.on('data', chunk => fileData += chunk.toString());
        req.on('end', () => {
            myEmitter.emit('fileUploaded'); // Trigger event
            res.end('File uploaded successfully');
        });
    }
});
server.listen(3000, () => console.log('Server running'));
```

MONGODB QUERIES

```
a) db.students.find({ age: { $gt: 18 } })
b) db.books.find({ title: { $regex: "MongoDB" } })
```

OUTPUT

> **Node.js:** Console prints Custom Event: File upload complete! upon POST request.
> **MongoDB:** Returns documents mapping to the criteria.

RESULT

The application successfully handled data streams and emitted custom events.

QUESTION 9

AIM

To split a web address into readable parts and display custom error messages using Node.js, and to sort MongoDB results.

PROCEDURE

1. Import the `url` module and use `url.parse(req.url, true)` to split the address.
2. Use `res.writeHead(404)` to send a custom error message if a route isn't found.
3. For MongoDB, chain `.sort({ age: -1 })` and `.limit(5)` to the `find()` query.

NODE.JS PROGRAM

```
const http = require('http');
const url = require('url');

const server = http.createServer((req, res) => {
    // Splits up a web address into readable parts
    const parsedUrl = url.parse(req.url, true);

    if (parsedUrl.pathname === '/info') {
        res.writeHead(200, {'Content-Type': 'application/json'});
        res.end(JSON.stringify(parsedUrl.query)); // Displays parsed URL parts
    } else {
        // Display customized error message
        res.writeHead(404, {'Content-Type': 'text/plain'});
        res.end('Error 404: The custom page you are looking for does not exist.');
    }
});
server.listen(3000, () => console.log('Server running'));
```

MONGODB QUERIES

```
a) db.students.find().sort({ age: -1 }).limit(5)
b) db.books.find({ title: { $regex: ".*MongoDB.*" } })
```

OUTPUT

```
Node.js: Browser displays parsed query object or custom Error 404 message.
MongoDB: Returns the 5 oldest students and books matching the regex.
```

RESULT

URLs were successfully parsed into query objects, and database results were correctly limited and sorted.

QUESTION 10

QUESTION 10

AIM

To write a Node.js program that passes query strings and displays the current date/time, and to update specific documents in MongoDB.

PROCEDURE

1. Create an Express/Node app that reads `req.query` to pass strings.
2. Use JavaScript's native `new Date()` object to return the current date and time.
3. For MongoDB, use `updateOne` based on customerID, and `updateMany` to add a new field to all documents.

NODE.JS PROGRAM

```
const express = require('express');
const app = express();

app.get('/time', (req, res) => {
    // Pass a query string in URL (e.g., ?user=John)
    const user = req.query.user || 'Guest';

    // Display the current date and time in the browser
    const currentDateTime = new Date().toLocaleString();

    res.send(`Hello ${user}, the current date and time is: ${currentDateTime}`);
});

app.listen(3000, () => console.log('Server running'));
```

MONGODB QUERIES

```
a) db.customers.updateOne({ customerID: 12345 }, { $set: { email: "newemail@test.com" } })
b) db.users.updateMany({}, { $set: { status: "inactive" } })
```

OUTPUT

```
Node.js: Hello John, the current date and time is: 4/8/2026, 7:08:55 PM.
MongoDB: { matchedCount: X, modifiedCount: Y }.
```

RESULT

Query strings were dynamically rendered alongside the current system time, and MongoDB effectively updated individual and bulk records.